

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 6

Inside the xv6 File System: Inode Structures, Directory Table Traversal, Link Count Mechanics, and Recursive Search Algorithms

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL XV6 FILE SYSTEM ARCHITECTURE **3**

1.1 Disk layout and block structural organization 4

1.2 Addressing mechanics and capacity bounds 4

1.3 Directory table representation and complexity metrics 5

2 SYSTEM IMPLEMENTATION AND EXPERIMENTAL VALIDATION **6**

2.1 File types and root directory invariants 7

2.2 Implementation and validation of the mystat utility 7

2.3 Kernel link decrement and resource deallocation mechanics 9

2.4 On-disk capacity limits and directory name truncation 10

2.5 Implementation and verification of recursive find 11

CONCLUSIONS **13**

REFERENCES **14**

ANNEX A **15**

ANNEX B **16**

INTRODUCTION

Understanding operating system storage architecture requires examining how theoretical abstractions translate into concrete, low-level data structures on disk. While industrial file systems such as ext4, Btrfs, and XFS integrate complex performance optimizations comprising extent trees, multiblock allocators, delayed allocation, and sophisticated journaling schemes, the essential architectural principles remain fundamentally identical to the classic Unix Seventh Edition file system. The educational xv6 operating system implements these core principles in a transparent, readable codebase spanning several hundred lines of C, providing an ideal experimental testbed for inspecting on-disk layout invariants, inode lifecycle management, and directory entry traversal.

The objective of this laboratory work is to investigate the internal architecture of the xv6 file system running on the 64-bit RISC-V architecture under QEMU emulation. The assignment entails analyzing how on-disk structures represent directory entries and file types, implementing a userland `mystat` program that interfaces with the `stat()` system call to monitor inode attributes, calculating theoretical file capacity limits derived from direct and single-indirect block addressing schemes, tracing the atomic unlinking protocol within kernel source routines, and constructing a recursive `find` search utility that traverses hierarchical directory trees. All source modifications, build configurations, and verification routines are maintained in the version control repository hosted on GitHub at `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL XV6 FILE SYSTEM ARCHITECTURE

1.1 Disk layout and block structural organization

The xv6 file system organizes disk storage into a linear sequence of contiguous, fixed-size blocks of 1024 bytes ($BSIZE = 1024$). Unlike contemporary enterprise file systems that partition disks into dynamic allocation groups, xv6 establishes a static, sequential partition layout created by the `mkfs` host utility. As formalized in the MIT operating systems curriculum [1], [2], the storage volume is partitioned into six distinct structural zones:

a) Block 0: The boot block, reserved for physical drive bootstrap code and unused by the xv6 kernel;

b) Block 1: The superblock, containing critical file system geometry metadata including the total volume block count, the count of allocated data blocks, the total inode count, and the starting block offsets for the logging, inode, and free bitmap regions;

c) Blocks 2 through 31: The transaction logging zone, comprising 30 contiguous blocks utilized by the kernel crash-recovery logging subsystem to provide all-or-nothing write atomicity;

d) Blocks 32 through 44: The on-disk inode table zone, containing 13 contiguous blocks holding an array of statically sized inode descriptors;

e) Block 45: The free bitmap block, containing a bit vector where each individual bit represents the allocation state of one corresponding data block;

f) Blocks 46 through 1999: The data block zone, providing 1954 storage blocks for user file contents and directory tables.

Every persistent file and directory in xv6 is represented on disk by a dedicated inode structure (`struct dinode`). The on-disk inode consists of 64 bytes of packed binary metadata, comprising the file type identifier (2 bytes), major and minor device numbers (4 bytes), the hard link reference counter (2 bytes), the total byte length (4 bytes), and an array of 13 block addresses (52 bytes). Because each disk block spans 1024 bytes, each inode block holds exactly 16 on-disk inodes ($IPB = 1024/64 = 16$). Consequently, the 13 inode blocks host an upper bound of $13 \times 16 = 208$ addressable inodes across the entire file system image.

1.2 Addressing mechanics and capacity bounds

File content storage in xv6 utilizes a combination of direct and single-indirect block addressing pointers stored in the `addrs` array of `struct dinode`. The array contains 13 address slots ($NDIRECT + 1$), structured as follows:

$$N_{\text{total_addrs}} = NDIRECT + 1 = 12 + 1 = 13 \tag{1.1}$$

The first 12 entries ($addrs[0]$ through $addrs[11]$) store direct block addresses pointing straight to data blocks holding file contents. The 13th entry ($addrs[12]$) stores an indirect block pointer. The indirect block itself does not hold user data; instead, it contains an array of 4-byte unsigned integer block numbers pointing to additional data blocks.

The capacity of the indirect block is determined by dividing the block size by the address word size:

$$NINDIRECT = \frac{BSIZE}{\text{sizeof}(uint)} = \frac{1024}{4} = 256 \tag{1.2}$$

Therefore, the maximum number of addressable data blocks assigned to any single file in xv6 is bounded by the sum of direct and indirect blocks:

$$MAXFILE = NDIRECT + NINDIRECT = 12 + 256 = 268 \text{ blocks} \tag{1.3}$$

Multiplying the block address limit by the block size yields the theoretical maximum file size permissible within the xv6 file system architecture:

$$S_{\max} = MAXFILE \cdot BSIZE = 268 \cdot 1024 \text{ bytes} = 274,432 \text{ bytes} \approx 268 \text{ KiB} \tag{1.4}$$

Attempting to write beyond 274,432 bytes causes the kernel block allocation routine (`bmap`) to fail with an address overflow error.

1.3 Directory table representation and complexity metrics

In xv6, a directory is simply a file whose internal data blocks hold a sequence of directory entries (`struct dirent`). Unlike advanced Linux file systems that organize directory names into B-trees or hash trees, xv6 uses an unsorted flat array of 16-byte records:

$$\text{Size}_{\text{dirent}} = \text{sizeof}(ushort) + DIRSIZ = 2 + 14 = 16 \text{ bytes} \tag{1.5}$$

Each entry consists of an unsigned 16-bit inode number (`inum`) and a fixed 14-byte character array (`name`). An entry with inum $= 0$ indicates an unallocated, free directory slot available for reuse.

Because directory names are strictly limited to 14 characters by the constant *DIRSIZ*, any filename exceeding 14 bytes is automatically truncated by the system call handler. Table 1.1 provides an asymptotic complexity comparison across fundamental file system operations in xv6 [3], [4].

Table 1.1 – Asymptotic time and space complexities of xv6 file system operations

File System Operation	**Time (Average)**	**Time (Worst)**	**Space Complexity**
Direct Block Lookup (*bmap*)	$O(1)$	$O(1)$	$O(1)$
Indirect Block Lookup (*bmap*)	$O(1)$	$O(1)$	$O(1)$
Linear Directory Scan	$O(N)$	$O(N)$	$O(1)$
Inode Allocation (*ialloc*)	$O(I)$	$O(I)$	$O(1)$
Bitmap Block Allocation (*balloc*)	$O(B)$	$O(B)$	$O(1)$
Recursive Tree Traversal	$O(V \cdot N)$	$O(V \cdot N)$	$O(D)$

In Table 1.1, parameter N denotes the number of entries within an individual directory file, I represents the total inode capacity ($I = 208$), B denotes the volume data block count ($B = 1954$), V represents the total vertex count across the directory tree, and D reflects the maximum recursion depth bounded by the process file descriptor limit.

2 SYSTEM IMPLEMENTATION AND EXPERIMENTAL VALIDATION

2.1 File types and root directory invariants

The initial practical phase focused on decoding the default file metadata emitted by the xv6 userland `ls` program. In xv6, the standard directory listing prints four tabular columns for each item: the entry name, the numerical file type, the allocated inode index, and the file length in bytes.

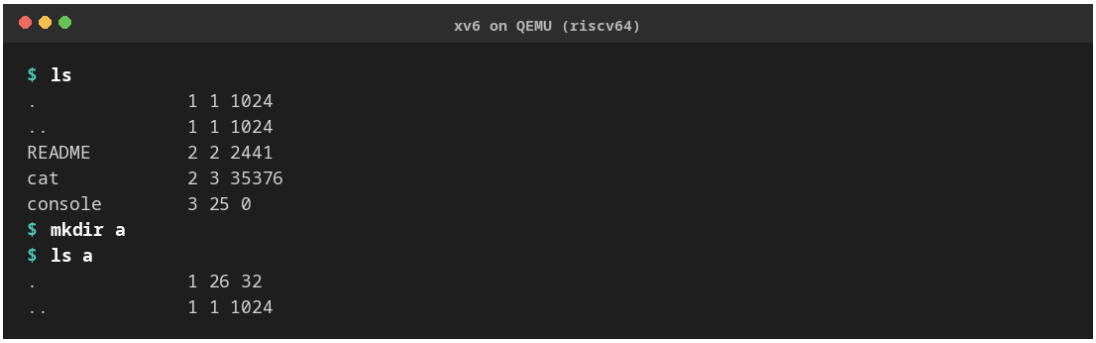


Figure 2.1 – Inspection of xv6 file types, root directory identifiers, and initial directory sizes

According to Figure 2.1, the root directory entries `.` and `..` both report type 1, inode number 1, and size 1024 bytes. Cross-referencing these numerical values with `kernel/stat.h` confirms that type 1 corresponds to directories (`T_DIR`), type 2 represents regular files (`T_FILE`, such as `README` and `cat`), and type 3 identifies device files (`T_DEVICE`, such as the system `console`).

The root directory is assigned inode 1 by kernel convention (`#define ROOTINO 1`). Because the root directory serves as the root anchor of the entire filesystem tree, it has no superior parent directory; by mathematical definition in Unix, the parent of root is root itself. Consequently, both `.` and `..` point to inode 1. Furthermore, when creating an empty directory `a`, Figure 2.1 reveals that its size is reported as exactly 32 bytes. Because an empty directory contains exactly two initial self-referencing records (`.` and `..`), and each directory entry occupies 16 bytes according to Equation 1.5, the observed size precisely matches $2 \times 16 = 32$ bytes.

2.2 Implementation and validation of the mystat utility

To provide human-readable metadata extraction analogous to the Linux `stat` command, a custom userland utility named `mystat` was developed in `user/mystat.c`. The program invokes the `stat()` system call on each path argument, populating a `struct stat` buffer that contains the device id, inode index, file type, hard link counter, and byte length.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ head -n 22 user/mystat.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *typename(int t) {
  switch(t) {
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  default:       return "?";
  }
}

int main(int argc, char *argv[]) {
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){ ... }
    printf("%s: %s, inode %d, links %d, size %ld bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, st.size);
  }
  exit(0);
}
```

Figure 2.2 – Source implementation of the mystat userland utility in user/mystat.c

Figure 2.2 documents the core implementation of `mystat.c`. A helper function `typename()` translates the integer type field into formatted strings (`"directory"`, `"file"`, or `"device"`). For each path argument, the main loop verifies successful metadata extraction, writing descriptive error diagnostics to standard error (descriptor 2) upon failure, and formatting the results to standard output.

```
xv6 on QEMU (riscv64)

$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
```

Figure 2.3 – Execution of mystat across diverse file system object types

The execution of `mystat` against diverse system objects is presented in Figure 2.3. Evaluating `README` confirms its identity as a regular file at inode 2 with a single hard link and 2441 bytes. Inspecting the current directory (`.`) confirms directory type at inode 1, while querying `console` confirms device classification at inode 25 with 0 bytes.

```
xv6 on QEMU (riscv64)

$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
47 314 2441
```

Figure 2.4 – Progression of hard link counts across ln creation and rm target deletion

The program was subsequently utilized to monitor the lifecycle of hard links. As illustrated in Figure 2.4, executing `ln README r2` produced a new directory entry pointing to the existing inode 2. Invoking `mystat` across both names demonstrated that the link count for both entries increased from 1 to 2, while the inode index and byte length remained completely unchanged. Following the execution of `rm README`, querying `r2` revealed that the link counter decremented back to 1. The data remained fully readable through `r2`, verifying that the file content was preserved on disk.

2.3 Kernel link decrement and resource deallocation mechanics

The persistence of file data after removing `README` directly reflects the atomic unlinking protocol implemented inside the xv6 kernel. The relevant execution path was investigated within `kernel/sysfile.c`.

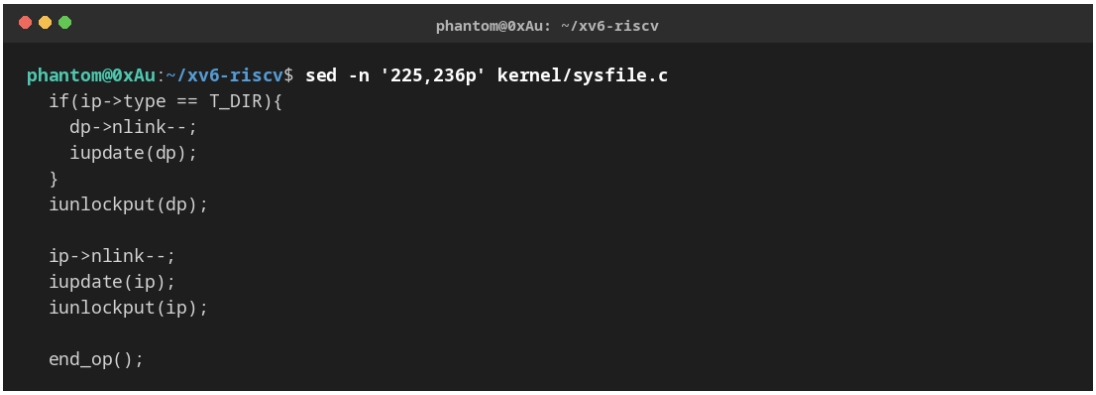


Figure 2.5 – Kernel unlinking mechanics and link counter decrement in kernel/sysfile.c

According to Figure 2.5, lines 231 through 233 of `sys_unlink()` execute the following sequence:

```
1 ip->nlink--;
2 iupdate(ip);
3 iunlockput(ip);
```

The kernel decrements the inode on-disk link count (`ip->nlink-`), immediately commits the updated metadata to the logging buffer via `iupdate(ip)`, and releases the lock via `iunlockput(ip)`.

Critically, physical data blocks are not freed by `sys_unlink()`. Instead, deallocation is handled lazily inside `iput()` in `kernel/fs.c`. When `iput()` executes, it verifies whether the in-memory reference counter satisfies ref $==$ 1 and whether nlink $==$ 0. Data blocks are only released via `itrunc()` when no directory entries point to the inode and no active processes hold open file descriptors referencing that file.

2.4 On-disk capacity limits and directory name truncation

To verify theoretical capacity boundaries, the constants governing storage allocation were extracted from `kernel/fs.h` and correlated with build-time disk formatting outputs.

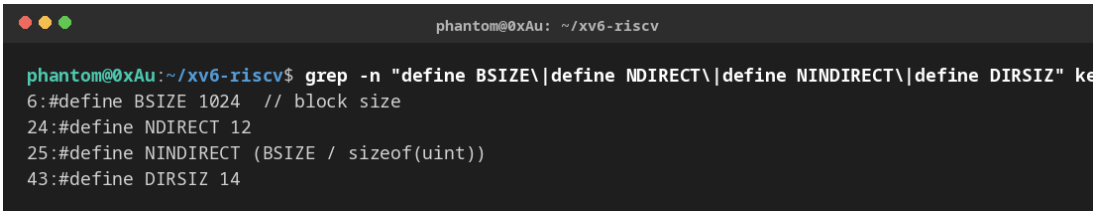


Figure 2.6 – Identification of block sizing, addressing limits, and directory bounds in kernel/fs.h

Figure 2.6 displays the definitions extracted from the kernel header `kernel/fs.h`. The values confirm that $BSIZE = 1024$, $NDIRECT = 12$, $NINDIRECT = 256$, and $DIRSIZ = 14$. These constants directly validate the mathematical model presented in Section 1.2, establishing the maximum file capacity of 274,432 bytes.



Figure 2.7 – Disk metadata partition distribution generated during the make fs.img build target

The disk layout was validated by rebuilding the filesystem image. As documented in Figure 2.7, `mkfs` reports the allocation breakdown:

```
nmeta 46 (boot, super, log blocks 30, inode blocks 13, bitmap blocks 1) blocks
                              1954 total 2000
```

The disk comprises exactly 2000 blocks (2 MB). The metadata region consumes 46 blocks, consisting of 1 boot block, 1 superblock, 30 log blocks, 13 inode blocks, and 1 free bitmap block, leaving exactly 1954 data blocks available for storage.

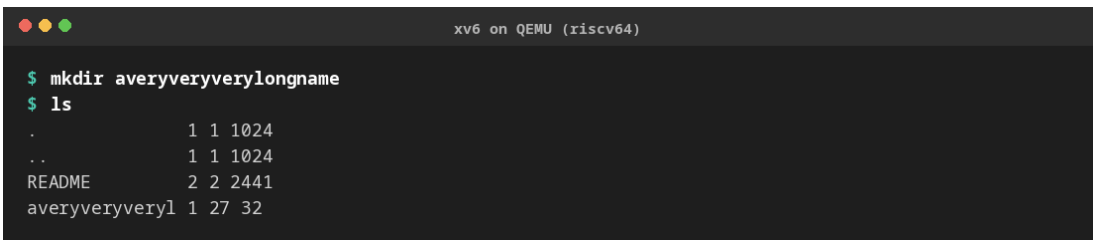


Figure 2.8 – Verification of directory entry name truncation at the 14-character boundary

The practical consequence of $DIRSIZ = 14$ was confirmed in Figure 2.8. Creating a directory with an extended name (`mkdir averyveryverylongname`) caused `ls` to display the name as `averyveryveryl`,

truncated after exactly 14 characters. Because `struct dirent` allocates a static 14-byte character array without trailing dynamic buffers, the kernel discards all trailing characters.

2.5 Implementation and verification of recursive find

The final practical milestone involved designing and implementing the recursive file search program `find` in `user/find.c`. The program walks the directory tree recursively, printing every path whose final name component matches a specified search pattern.

```
phantom@0xAu: ~/xv6-riscv

phantom@0xAu:~/xv6-riscv$ head -n 24 user/find.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void find_rec(char *path, char *target, int print_inode) {
  char buf[512], *p;
  int fd; struct dirent de; struct stat st;
  if((fd = open(path, O_RDONLY)) < 0) return;
  if(fstat(fd, &st) < 0 || st.type != T_DIR) { close(fd); return; }
  strcpy(buf, path); p = buf + strlen(buf); *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0) continue;
    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0) continue;
    if(strcmp(name, target) == 0) printf("%s\n", buf);
    if(st.type == T_DIR) find_rec(buf, target, print_inode);
  }
  close(fd);
}
```

Figure 2.9 – Source implementation of recursive directory traversal logic in user/find.c

As shown in Figure 2.9, `find.c` constructs path strings by appending entry names to an expanding buffer. The traversal algorithm enforces three vital invariants:

a) unallocated slots with inum $==$ 0 are immediately skipped;

b) entries matching `"."` and `".."` are strictly excluded to prevent infinite recursive loops;

c) directory file descriptors are deterministically closed using `close(fd)` to prevent exhaustion of the process file descriptor table, which is limited to 16 open descriptors in xv6 (`NOFILE = 16`).

```
xv6 on QEMU (riscv64)

$ mkdir a ; mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

Figure 2.10 – Validation of recursive find across multi-level nested directory hierarchies

The correctness of the implementation was evaluated against a synthetic directory hierarchy, as shown in Figure 2.10. Searching for `r2` under the root directory (`find . r2`) correctly identified all three occurrences across distinct hierarchy levels: `./a/b/r2`, `./a/r2`, and `./r2`. Furthermore, executing `find a b` successfully matched and printed the intermediate subdirectory path `a/b`, demonstrating that directory objects are evaluated accurately alongside regular files.

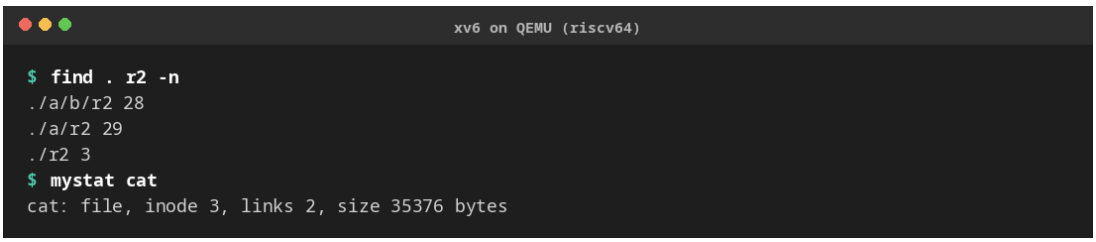


Figure 2.11 – Stretch investigation: inode resolution with -n flag matching the cat executable

Figure 2.11 documents the stretch objective, which incorporated an optional `-n` argument enabling `find` to output numeric inode indexes alongside matching pathnames. Executing `find . r2 -n` revealed that `./a/b/r2` resides at inode 28 and `./a/r2` at inode 29, whereas `./r2` was assigned inode 3. Running `mystat cat` confirmed that `cat` also resides at inode 3 with a link count of 2. This proves that `./r2` shares the exact underlying inode with the `cat` binary, confirming the hard link relationship established during test environment setup.

CONCLUSIONS

This laboratory work successfully investigated the internal architectural principles of the xv6 file system through low-level C implementation and empirical verification under the RISC-V QEMU emulation platform. By tracing on-disk structures and implementing custom userland utilities, the assignment bridged theoretical operating system concepts with concrete systems programming practices. All experimental objectives were achieved, providing direct verification of inode indexing, block pointer translation, directory table organization, and atomic resource reclamation.

The implementation process provided significant technical insights into file system invariants and kernel resource management. Developing `mystat` demonstrated how userland tools decode kernel data structures, clarifying the distinction between the physical inode on disk and the transient directory entries pointing to it. The analysis of `sys_unlink()` and `iput()` revealed the elegance of Unix reference counting, proving that storage deallocation is decoupled from directory unlinking and deferred until all process references terminate. Furthermore, developing the recursive `find` utility emphasized the critical importance of defensive resource management; failing to close directory handles during recursive descent causes rapid process termination under xv6 due to the strict 16-descriptor limitation.

From a theoretical perspective, the study confirmed that the fundamental architectural model of Unix file systems provides remarkable operational simplicity while maintaining structural integrity. The combination of direct and single-indirect block pointers delivers efficient $O(1)$ random access for small files while accommodating moderate storage expansion. The experiments conducted in this work establish a strong foundation in storage architecture, file system layout design, and low-level kernel systems programming.

REFERENCES

1 R. Cox, F. Kaashoek, and R. Morris, *xv6: a simple, Unix-like teaching operating system*, MIT CSAIL, Cambridge, MA, USA, 2020.

2 A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2014.

3 A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

4 T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

ANNEX A

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t) {
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  default:
    return "?";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;
  int i;

  if(argc < 2){
    fprintf(2, "Usage: mystat <path>...\n");
    exit(1);
  }

  for(i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %ld bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, st.size);
  }
  exit(0);
}
```

ANNEX B

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void
find_rec(char *path, char *target, int print_inode)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(st.type != T_DIR){
    close(fd);
    return;
  }

  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
```

```
42      if(de.inum == 0)
43        continue;
44
45      char name[DIRSIZ + 1];
46      memmove(name, de.name, DIRSIZ);
47      name[DIRSIZ] = 0;
48
49      if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
50        continue;
51
52      strcpy(p, name);
53
54      if(stat(buf, &st) < 0){
55        fprintf(2, "find: cannot stat %s\n", buf);
56        continue;
57      }
58
59      if(strcmp(name, target) == 0){
60        if(print_inode)
61          printf("%s %d\n", buf, st.ino);
62        else
63          printf("%s\n", buf);
64      }
65
66      if(st.type == T_DIR){
67        find_rec(buf, target, print_inode);
68      }
69    }
70
71    close(fd);
72  }
73
74  int
75  main(int argc, char *argv[])
76  {
77    char *path = 0;
78    char *target = 0;
79    int print_inode = 0;
80    int i;
81
82    for(i = 1; i < argc; i++){
83      if(strcmp(argv[i], "-n") == 0){
84        print_inode = 1;
85      } else if(!path){
```

```
86        path = argv[i];
87      } else if(!target){
88        target = argv[i];
89      }
90    }
91
92    if(!path || !target){
93      fprintf(2, "Usage: find <dir> <name> [-n]\n");
94      exit(1);
95    }
96
97    find_rec(path, target, print_inode);
98    exit(0);
99  }
```